Sesion 2: Automatizacion en AWS

CodeBuild · CodePipeline · AWS Inspector — del codigo a la nube con calidad integrada

Certificacion 5: QA Cloud-Native - Automatizacion Inteligente con IA Generativa

Enfoque del dia | El pipeline CI/CD | Caso 1: Pipeline local | Caso 2: CodeBuild | Caso 3: CodePipeline | Caso 4: AWS Inspector | A tener en cuenta

Enfoque

De "los tests corren en mi maquina" a "los tests corren solos en la nube cada vez que alguien hace push"

Dia 1 al Dia 2

Antes generamos datos de prueba de calidad. Hoy los ponemos en marcha: conectamos nuestros tests a un **pipeline de CI/CD en AWS**. Cada vez que alguien sube codigo, el pipeline instala dependencias, levanta la API, corre los tests y reporta el resultado — sin que nadie apriete un boton. Si los tests fallan, el deploy se detiene.

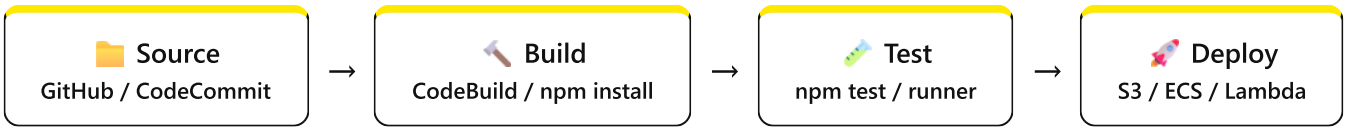
Enfoque hibrido: local primero, nube despues

Antes de subir a AWS, corremos el pipeline **en nuestra maquina** usando el mismo `buildspec.yml` que usara CodeBuild. Asi entendemos cada fase, vemos el resultado en directo, y cuando lo subimos a la nube no hay sorpresas.

Bloque	Tema	Herramienta
1	Generar datos (Faker) + servicio + pipeline local (4 fases)	Faker + json-server + runner-local.js
2	Pipeline en la nube: CodeBuild	AWS CodeBuild + buildspec.yml
3	Orquestacion: CodePipeline (Source → Build → Test)	AWS CodePipeline + GitHub/CodeCommit
4	Seguridad: escaneo de vulnerabilidades	AWS Inspector v2

El pipeline CI/CD: anatomia

Que ocurre desde que alguien hace git push hasta que el codigo esta en produccion



QA en el pipeline

Un test que solo corre cuando el desarrollador se acuerda no es automatizacion: es una sugerencia. El pipeline hace que los tests sean **obligatorios**: si fallan, el codigo no llega a produccion. Es la diferencia entre "tenemos tests" y "los tests protegen el producto".

El buildspec.yml: el guion del pipeline

Es un archivo YAML que le dice a CodeBuild exactamente que hacer en cada fase. Vive **dentro del repositorio** junto al codigo, se versiona con Git, y cualquiera del equipo puede leerlo y entender que hace el pipeline. Cuatro fases: `install`, `pre_build`, `build`, `post_build`.

buildspec.yml — estructura de las 4 fases

```
version: 0.2
phases:
  install:       # instala dependencias (npm install, pip install, etc.)
    commands:
      - npm install

  pre_build:     # verifica el entorno ANTES de correr tests
    commands:
      - node 1-generar-datos/generar-datos.js  # Faker crea el dataset
      - node 4-pipeline/restaurar-seed.js       # dataset limpio
      - npx json-server 2-servicio/db.json --port 3001 &
      - sleep 3                                 # esperar que levante

  build:         # corre los tests — exit 1 = BUILD FAILED
    commands:
      - npm test

  post_build:    # siempre corre (incluso si build fallo)
    commands:
      - cat 4-pipeline/test-report.json   # mostrar reporte

artifacts:       # archivos que se suben a S3
  files:
    - 4-pipeline/test-report.json
```

💡 La clave esta en el **codigo de salida**: `process.exit(1)` en el runner hace que CodeBuild marque el build como FAILED y CodePipeline detiene el deploy. Sin ese exit code, el pipeline no sabe si los tests pasaron.

Caso practico 1: Pipeline local (las 4 fases en tu maquina)

Entiende el pipeline antes de subirlo a AWS

El servicio bajo prueba: servicio-tienda

Una API con dos recursos: **productos** (10 items con precio, stock, categoria, activo) y **pedidos** (5 pedidos con estado, cliente, cantidad). Lo primero que hacemos NO es escribir esos datos a mano: los **generamos con Faker** (igual que en el Dia 1), con semilla fija para que sean reproducibles. Luego tres suites de tests los validan: productos (contrato + reglas de negocio + filtros), pedidos (contrato + CRUD completo), y contrato/rendimiento (Content-Type, SLA <800ms, paginacion, busqueda, orden). El pipeline lo corre todo en secuencia.

Estructura del proyecto (carpetas numeradas por paso)

```
servicio-tienda/
├── 1-generar-datos/
│   └── generar-datos.js        ← PASO 1: Faker crea el dataset (reproducible)
├── 2-servicio/                 (vacia al clonar: los datos se generan)
│   ├── db.seed.json            ← estado original (lo crea el paso 1, NO versionado)
│   └── db.json                 ← copia de trabajo que lee la API (se genera en el paso 2)
├── 3-tests/
│   ├── test-productos.js       ← Suite 1: contrato + reglas de negocio + filtros
│   ├── test-pedidos.js         ← Suite 2: contrato + ciclo CRUD
```

```
│   ├── test-contrato.js        ← Suite 3: SLA, paginacion, busqueda, orden
│   └── runner.js               ← orquesta las 3 suites, genera reporte
├── 4-pipeline/
│   ├── buildspec.yml           ← guion de CodeBuild (5 fases + security gate)
│   ├── runner-local.js         ← ejecuta las 5 fases en Node (comando oficial)
│   ├── runner-local.ps1        ← alternativa PowerShell (opcional)
│   └── restaurar-seed.js       ← copia db.seed.json -> db.json
└── 5-inspector/
    ├── reporte-inspector.json  ← reporte de vulnerabilidades (simulado)
    └── analizar-reporte.js     ← interpreta el reporte, genera score
```

✏️ **EJERCICIO PRACTICO**

Ejercicio 1: generar los datos y correr el pipeline local

1. Terminal en `proyecto/servicio-tienda`. Instala dependencias: `npm install` (Faker + json-server). Debe decir "added X packages".

2. **Genera los datos con Faker:** `npm run generar`. Crea `2-servicio/db.seed.json` con 10 productos y 5 pedidos. Con la semilla fija, los datos son los mismos en cada corrida. Abrelo y observa el producto 10 (inactivo, stock 0): un caso de borde a proposito.

3. Corre el pipeline local: `npm run pipeline`. Observa las fases INSTALL → PRE_BUILD → BUILD → POST_BUILD con colores. En PRE_BUILD el pipeline vuelve a generar los datos y restaura el seed. Al final aparece una 5ª fase, **SECURITY_GATE**: la analizamos a fondo en el Caso práctico 4.

4. Lee el resultado final: **PIPELINE PASSED** (o BUILD FAILED si algún test falla). Abre `4-pipeline/test-report.json`: contiene tests pasados, fallados y duracion.

5. Reto: rompe un test a proposito. Abre `2-servicio/db.json` y cambia el precio del producto 1 a `-99`. Corre `npm test` (con la API arriba). El test deberia fallar (precio no puede ser negativo). Restaura con `npm run seed`.

6. Mira el `4-pipeline/buildspec.yml` y localiza cada comando que ejecuto el runner local. Son exactamente los mismos que correria AWS CodeBuild en la nube.

Por que empezar generando los datos

Conecta con el Dia 1: los datos de prueba no se escriben a mano, se generan. Con Faker y una semilla fija tenemos datos realistas Y reproducibles. El pipeline los regenera en cada corrida, asi cualquier persona del equipo (o el contenedor de CodeBuild) arranca desde el mismo dataset, sin depender de un JSON que alguien edito.

Por que primero local

Cada minuto de build en CodeBuild cuesta dinero y tarda 2-3 minutos en arrancar el contenedor. Depurar en local es gratis e instantaneo. La regla: **el pipeline pasa local antes de subir a AWS**. Esta regla aplica en cualquier herramienta de CI/CD (GitHub Actions, Jenkins, Azure Pipelines).

💡 `npm run pipeline` usa `runner-local.js` (Node puro, sin dependencias de PowerShell). Si el proceso json-server queda vivo tras un fallo a mitad, liberalo: `Get-Process node | Stop-Process -Force`. El `runner-local.ps1` sigue disponible si prefieres correrlo directamente desde PowerShell: `powershell -ExecutionPolicy Bypass -File 4-pipeline/runner-local.ps1`.

Caso practico 2: AWS CodeBuild

El mismo pipeline, en un contenedor en la nube de AWS

Que es CodeBuild

Un servicio de build **completamente administrado** de AWS. No hay servidor que mantener: AWS aprovisiona un contenedor (imagen administrada con Node 20, ej. `amazonlinux-x86_64-standard:6.0`), corre tu `buildspec.yml` y lo destruye al terminar. Pagas solo por los minutos de build. El reporte de tests queda en **CloudWatch Logs** y los artefactos en **S3**.

Concepto	Local (runner-local.js)	AWS CodeBuild
Donde corre	Tu maquina	Contenedor en AWS (efimero)
Como se configura	runner-local.js (Node puro)	buildspec.yml (mismo archivo)
Logs	Consola Node (colores ANSI)	CloudWatch Logs
Artefactos	4-pipeline/test-report.json local	S3 bucket
Se activa	Tu manualmente	Automatico: push a git / webhook
Costo	Gratis	~$0.005/min (Free Tier: 100 min/mes)

Paso previo obligatorio: el codigo tiene que estar en GitHub

CodeBuild NO lee tu maquina: lee un repositorio. Antes de crear el proyecto en AWS, hay que subir `servicio-tienda` a GitHub. Lo que se sube es el **codigo fuente** (los scripts, los tests, el buildspec) — NO los datos ni `node_modules`: esos se generan en la nube durante el build (por eso estan en el `.gitignore`).

Que se sube a GitHub y que NO

```
SI se sube (codigo fuente):        NO se sube (lo ignora .gitignore):
  1-generar-datos/generar-datos.js   node_modules/       (npm install lo baja)
  2-servicio/         (carpeta)      2-servicio/db.seed.json  (Faker lo genera)
  3-tests/*.js                       2-servicio/db.json       (se genera)
  4-pipeline/buildspec.yml           4-pipeline/test-report.json (se genera)
  4-pipeline/runner-local.js         5-inspector/resumen-analisis.json
  5-inspector/analizar-reporte.js
  5-inspector/reporte-inspector.json
  package.json  package-lock.json
  .gitignore  README.md
```

🪛 EJERCICIO PRACTICO

Ejercicio 2A: subir el proyecto a GitHub

1 En github.com/new crea un repositorio nuevo llamado `servicio-tienda`. Dejalo vacio (sin README ni .gitignore, ya los tenemos). Copia la URL: `https://github.com/TU-USUARIO/servicio-tienda.git`.

2 Terminal en `dia2/proyecto/servicio-tienda`. Inicializa git y confirma que el `.gitignore` existe (para no subir node_modules ni datos):

PowerShell — en servicio-tienda/

```
git init
git branch -M main
git add .
git status              # revisa: NO deben aparecer node_modules ni db.json
git commit -m "Servicio tienda: pipeline QA para CodeBuild"
```

3 Conecta tu repo remoto y sube el codigo:

PowerShell

```
git remote add origin https://github.com/TU-USUARIO/servicio-tienda.git
git push -u origin main
```

4 Recarga tu repo en GitHub. Confirma que ves las carpetas `1-generar-datos` … `5-inspector` y el `package.json`, pero **NO** ves `node_modules` ni archivos de datos. Si ves node_modules, revisa el `.gitignore`.

💡 Si git te pide autenticacion al hacer push, GitHub ya no acepta contrasena: usa un **Personal Access Token** (github.com → Settings → Developer settings → Personal access tokens) como contrasena, o instala **GitHub CLI** (`gh auth login`) que lo gestiona por ti.

✏️ **EJERCICIO PRACTICO**

Ejercicio 2B: crear el proyecto en CodeBuild (demo guiada)

1 Consola AWS → **CodeBuild** → **Create build project**. Project name: `servicio-tienda-tests`.

2 **Source:** Source provider = **GitHub**. "Connect to GitHub" (crea/autoriza la conexion GitHub App la primera vez y dale acceso al repo). Repository: selecciona `<usuario>/servicio-tienda`. Reference type: Branch → `main`.

3 **Environment:** Managed image → Operating system: **Amazon Linux** → Runtime: Standard → Image: `aws/codebuild/amazonlinux-x86_64-standard:6.0` (trae Node 20). Service role: New service role (deja el nombre por defecto). En **Registros**: marca CloudWatch Logs para poder ver el reporte.

4 **Buildspec:** "Use a buildspec file" → Buildspec name: `4-pipeline/buildspec.yml`. Esta es la ruta EXACTA dentro del repo; si el buildspec estuviera en la raiz seria solo `buildspec.yml`.

5 **Artifacts:** Type = Amazon S3 → crea o elige un bucket `tienda-test-reports-<usuario>`. (Opcional; sin esto los logs igual salen en CloudWatch.) Click **Create build project**.

6 En la pagina del proyecto: **Start build**. Observa la pestana **Build logs** en tiempo real: veras las 4 fases (INSTALL → PRE_BUILD → BUILD → POST_BUILD), las mismas que en local. Al terminar: **Succeeded** en verde.

Lo que ves en los Registros de compilacion durante el build

```
[Container] Phase: INSTALL
  npm install → added 121 packages

[Container] Phase: PRE_BUILD
  Generados 10 productos y 5 pedidos.   (Faker)
  Dataset restaurado: 10 productos, 5 pedidos.

[Container] Phase: BUILD
  API lista (el runner la levanta solo)
  Productos: 111 pasaron 0 fallaron
  Pedidos:    61 pasaron 0 fallaron
  Contrato:   14 pasaron 0 fallaron
  ✓  BUILD PASSED - todas las pruebas pasaron

[Container] Phase: POST_BUILD
  ===== FASE POST_BUILD: REPORTE DE TESTS =====
  { "pasadas": 186, "falladas": 0, "exitCode": 0 }

BUILD SUCCEEDED
```

Que se espera ver al terminar el Ejercicio 2B

Cuando el build termina bien, deberias poder confirmar estos cuatro resultados. Si alguno falta, revisa la tabla de errores comunes de abajo.

Donde mirar	Resultado esperado
Estado de la compilacion	**Succeeded / Correcto** en verde · Numero de compilacion 1
Detalles de la fase	Todas en verde: SUBMITTED → QUEUED → PROVISIONING → DOWNLOAD_SOURCE → INSTALL → PRE_BUILD → BUILD → POST_BUILD → FINALIZING → COMPLETED
Registros de compilacion	En BUILD: **186 pasaron, 0 fallaron**. En POST_BUILD: el JSON con `"pasadas": 186, "falladas": 0`
Bucket S3 (si lo configuraste)	En `tiendatestreports/report/`: el `test-report.json` subido y descargable

Si algo sale en rojo	Que significa / donde mirar
DOWNLOAD_SOURCE	Repo mal conectado o error de YAML en el buildspec
INSTALL	Problema con npm / package.json
BUILD	Un test fallo (esperado si rompes un dato a proposito). El log muestra cual y por que
Registros vacios	Falto marcar **CloudWatch Logs** al crear el proyecto

💡 El nombre de la fase que sale en rojo te dice exactamente donde mirar. Ese es el valor de un pipeline por fases: el fallo esta localizado, no tienes que adivinar.

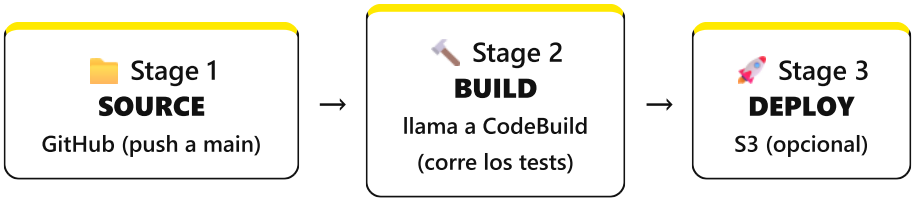
Caso practico 3: AWS CodePipeline

El orquestador: conecta el repositorio, el build y el deploy en un flujo automatico

CodeBuild vs CodePipeline

CodeBuild sabe correr un buildspec. **CodePipeline** sabe *cuando* llamar a CodeBuild y *que hacer despues*. CodePipeline conecta las piezas: "cuando haya un push en la rama main, llama a CodeBuild; si pasa, despliega a S3/ECS/Lambda; si falla, notifica por SNS/email". Es el director de orquesta.

⚠️ **Requisito previo:** este ejercicio REUTILIZA el proyecto de CodeBuild `servicio-tienda-tests` que creaste en el Ejercicio 2B. CodePipeline no corre tests por si mismo: en su stage de Build **llama a ese proyecto de CodeBuild**. Si no lo tienes creado, vuelve al Ejercicio 2B primero.



💡 Ojo: los tests NO son un stage aparte. Corren **dentro** del stage BUILD, porque el proyecto CodeBuild ya ejecuta el `buildspec.yml` completo (install → pre_build → build → post_build). CodePipeline solo orquesta: Source → Build → Deploy.

Antes de empezar: la conexion a GitHub (GitHub App)

Tanto CodeBuild como CodePipeline hablan con GitHub a traves de una **conexion de tipo GitHub App** (AWS Developer Tools → Connections). La primera vez tienes que crearla: **Crear conexion** → **GitHub** → **autorizar**, y en el instalador de la GitHub App darle acceso al repositorio `<usuario>/servicio-tienda` (o "All repositories"). Si ya la creaste en el Ejercicio 2B, reutiliza esa misma conexion aqui.

Ojo con el webhook: con GitHub App el disparador NO aparece en GitHub → Settings → Webhooks (eso es normal). La GitHub App usa su propio canal. Si un push no dispara el pipeline, revisa que la App tenga acceso al repo en *Connections* y que la conexion este en estado **Available**, no Pending.

🖍️ **EJERCICIO PRACTICO**

Ejercicio 3: crear el pipeline en CodePipeline (paso a paso)

1. **Abrir CodePipeline.** Consola AWS → "CodePipeline" → Canalizaciones → **Crear la canalizacion**.

2. **Elegir la opcion de creacion.** AWS muestra plantillas (ECR, ECS Fargate, CloudFormation, Terraform). NO elijas ninguna: en Category selecciona **"Crear una canalizacion personalizada"** → Siguiente. (Las plantillas son para desplegar contenedores/infra, no aplican aqui.)

3. **Configuracion.** Nombre: `tienda-pipeline`. Modo de ejecucion: **En cola**. Rol de servicio: **Nuevo rol** (deja marcado "Permitir que AWS CodePipeline cree un rol"). Siguiente.

4. **Etapa de origen (Source).** Proveedor: **GitHub (a traves de GitHub App)**. Conexion: la que ya autorizaste. **Nombre del repositorio:** `<usuario>/servicio-tienda` (formato `usuario/repo`, NO la URL). Rama: `main`. Webhook push/pull activado. Siguiente.

5. **Etapa de compilacion (Build).** Proveedor: **Otros proveedores** → **AWS CodeBuild**. Nombre del proyecto: `servicio-tienda-tests` (el del Ejercicio 2B). Tipo: Compilacion unica. Artefacto de entrada: SourceArtifact. Siguiente.

6. **Etapa de prueba (opcional): Omitir la etapa de prueba** (los tests ya corren dentro del Build). **Etapa de implementacion (opcional): Omitir la etapa de implementacion** (no desplegamos hoy).

7. **Revisar y crear.** Revisar → **Crear la canalizacion**. Apenas se crea, el pipeline arranca solo: SOURCE → BUILD se ponen en verde.

⚠️ **Error mas comun (y como evitarlo):** en "Nombre del repositorio" escribe `usuario/repo` (ej. `<usuario>/servicio-tienda`), **NO** la URL completa `https://github.com/<usuario>/servicio-tienda.git`. Si pones la URL, el stage Source falla con: "*No Branch [main] found for FullRepositoryName*". No es que falte la rama: AWS no puede identificar el repo con la URL, solo con el formato usuario/repo.

Como validar y probar el pipeline despues de creado

Crear el pipeline no basta: hay que probar que funciona como una compuerta de calidad. Tres validaciones:

🖍️ **EJERCICIO PRACTICO**

Validar el pipeline (3 pruebas)

1. **Camino feliz:** confirma Source ✓ y Build ✓ en verde. Click en **Details** del Build → Registros de compilacion → verifica "186 pasaron 0 fallaron". El verde es la consecuencia; el log del Build es la evidencia.

2. **Disparo automatico (esto ES el CI):** en tu terminal haz un cambio y sube: `echo "test" >> README.md` → `git add .` → `git commit -m "test disparo"` → `git push`. Vuelve a CodePipeline SIN tocar nada: el pipeline arranca solo en segundos. El webhook de GitHub lo disparo.

3. **Bloqueo de codigo malo (lo que protege):** rompe un test a proposito (edita una asercion en `3-tests/test-productos.js`), commit y push. Observa: Source verde → **Build ROJO**. El codigo malo se queda atrapado, no avanza. Arregla, push, y vuelve a verde.

Validacion	Resultado esperado
Camino feliz	**Source ✓** → **Build ✓** · Build muestra 186 tests OK
Disparo automatico	Tras un `git push`, el pipeline arranca **solo** en segundos
Bloqueo de codigo malo	Un test roto → **Build ROJO** · el pipeline se detiene

El poder del pipeline: feedback loop rapido

Sin pipeline: el bug llega a produccion horas o dias despues de que alguien lo introdujo. Con pipeline: el bug se detecta en **minutos**, antes de que el codigo salga del repositorio. Si el stage BUILD falla (un test roto), lo que sigue nunca corre: el codigo malo no llega a produccion.

💡 Recuerda: los tests corren DENTRO del stage Build (via el buildspec de CodeBuild), por eso omitimos la "etapa de prueba" de CodePipeline. Notificaciones: en CodePipeline → Notify → Create notification rule puedes mandar un email por SNS cuando un stage falla.

Caso practico 4: AWS Inspector v2

Seguridad como parte del pipeline: escaneo de vulnerabilidades automatico

Que es AWS Inspector v2

Un servicio de escaneo de vulnerabilidades que analiza: **instancias EC2** (OS + software), **imagenes de contenedor en ECR** (dependencias de la app), y **funciones Lambda** (librerias incluidas). Genera hallazgos con severidad CRITICO/ALTO/MEDIO/BAJO, numero CVE, version afectada, version que lo corrige y recomendacion. Se activa con un click y escanea continuamente.

Severidad	CVSS	Que significa	Tiempo de respuesta
CRITICO	9.0 – 10.0	Ejecucion remota de codigo, acceso root	Parchear HOY
ALTO	7.0 – 8.9	Escalada de privilegios, DoS severo	Parchear esta semana
MEDIO	4.0 – 6.9	Exposicion de datos, ReDoS, redireccion	Proxima iteracion
BAJO	0.1 – 3.9	Riesgo menor, imagen desactualizada	Backlog

⚠️ **Este caso NO es aislado:** es la 5ª fase del **mismo pipeline** del Ejercicio 2B/3. Igual que el BUILD corre los 186 tests, ahora el **SECURITY_GATE** corre el analisis de Inspector. Los tests dicen si la logica funciona; el gate dice si las dependencias son seguras. Misma tuberia, dos compuertas.

🖍️ **EJERCICIO PRACTICO**

Ejercicio 4: el security gate dentro del pipeline

1. Desde `proyecto/servicio-tienda` : `npm run inspector` . Corre el analisis suelto: dashboard de severidades, lista priorizada y plan de accion top 5. Es el mismo script que el pipeline invoca en su fase de seguridad.

2. Lee el **score de riesgo** al final: formula critico×10 + alto×5 + medio×2 + bajo×1. Con los 12 hallazgos del reporte el score es **50** → supera el umbral de 25 → **DEPLOY BLOQUEADO**.

3. Ahora corre el pipeline completo: `npm run pipeline` . Al final del BUILD veras la fase **SECURITY_GATE** que ejecuta el mismo analisis. Por defecto es **informativo**: avisa del score pero deja pasar (PIPELINE PASSED).

4. Activa el gate como **bloqueante** y vuelve a correr:
`$env:SECURITY_GATE="on"; npm run pipeline` (PowerShell)
Ahora los 186 tests pasan pero el pipeline termina en **SECURITY GATE FAILED** (exit 1): el mismo mecanismo que un test roto, pero por seguridad.

5. Reto: en `analizar-reporte.js`, cambia `UMBRAL_BLOQUEO` de 25 a 60. Corre `npm run pipeline` con `SECURITY_GATE=on`: el deploy ya no se bloquea (score 50 < 60). Discusion: ¿cuanto es demasiado permisivo?

resumen-analisis.json (generado por analizar-reporte.js)

```
{
  "score": 50,
  "nivelRiesgo": "ALTO - DEPLOY BLOQUEADO",
  "deployBloqueado": true,
  "resumen": { "total": 12, "critico": 2, "alto": 4, "medio": 4, "bajo": 2 },
  "top5": [
    { "id": "CVE-2021-44228", "severidad": "CRITICO", "paquete": "log4j-core",    "titulo": "Log4Shell..." },
    { "id": "CVE-2022-22965", "severidad": "CRITICO", "paquete": "spring-webmvc", "titulo": "Spring4Shell..." },
    ...
  ]
}
```

Como quedo integrado en NUESTRO pipeline

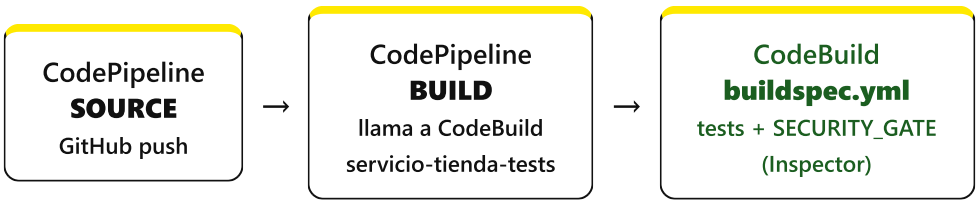
El analisis de Inspector no es un script suelto: es una fase mas del pipeline que ya construimos. En el `4-pipeline/runner-local.js` (y en el `buildspec.yml` de CodeBuild) hay una fase **SECURITY_GATE** despues del BUILD que ejecuta `analizar-reporte.js`. La variable `SECURITY_GATE` decide el comportamiento:

- `off` (por defecto): el gate es **informativo**, muestra el score pero no rompe el build. Asi los 186 tests siguen pasando en verde.
- on: el gate es **bloqueante**. Si el score supera el umbral, el pipeline termina en FAILED (exit 1), igual que un test funcional roto.

✏️ **EJERCICIO PRACTICO**

Ejercicio 4B: activar el gate en el CodePipeline del Ejercicio 3

Pregunta clave: **¿como se une esto al CodePipeline que ya cree?** La respuesta es que **no se toca CodePipeline**. El gate vive dentro del proyecto de CodeBuild que el pipeline ya invoca en su stage Build. Solo hay que activarlo ahi.



El buildspec ya corre `analizar-reporte.js` en su fase `post_build`. Si el gate esta en `on` y el score supera el umbral, CodeBuild devuelve BUILD FAILED, y CodePipeline detiene el pipeline solo. La union ya existe: solo falta encender el interruptor.

1. Consola AWS → **CodeBuild** → proyecto `servicio-tienda-tests` → **Editar** → **Entorno**.

2. Despliega **Configuracion adicional** → **Variables de entorno**. Agrega: Nombre `SECURITY_GATE`, Valor `on`. Guarda (Actualizar proyecto).

3. Dispara el pipeline: haz un `git push` (o en CodePipeline pulsa **Publicar cambio / Release change**). El stage Source pasa, el stage Build llama a CodeBuild.

4. Mira los logs del Build: tras los 186 tests corre el analisis de Inspector. Como el score es 50 > 25, CodeBuild termina en **BUILD FAILED** y el stage Build de CodePipeline se pone **en rojo**. El pipeline se detiene: no avanza a etapas siguientes.

5. Para volver a verde: cambia la variable a `off` (gate informativo) o baja el `UMBRAL_BLOQUEO` en `analizar-reporte.js`, haz push y vuelve a correr.

> 💡 Esto es exactamente lo mismo que corriste en local con `$env:SECURITY_GATE="on"; npm run pipeline`. La unica diferencia es DONDE se define la variable: en local es tu terminal; en AWS es la configuracion del proyecto de CodeBuild. El mecanismo (exit 1 → pipeline detenido) es identico.

La otra ruta: Inspector v2 nativo sobre ECR (nivel produccion)

La integracion anterior mete el analisis DENTRO del build (rapido, reproducible, ideal para aprender). La ruta "productiva" de AWS es distinta y mas pesada: Inspector v2 escanea **imagenes de contenedor en ECR** de forma continua. El flujo seria:

1. Contenerizar la app (un `Dockerfile`) y que el pipeline construya la imagen.
2. Un stage extra empuja la imagen a un repositorio **ECR** con "scan on push" activado.
3. Inspector v2 escanea la imagen automaticamente y publica los hallazgos.
4. Un **Lambda** (o un comando en el buildspec via AWS CLI) lee los hallazgos con `inspector2 list-findings`, calcula el score y devuelve exit 0/1 — el mismo gate, pero con datos reales de Inspector en vez de un reporte de ejemplo.

No hacemos esta ruta en clase porque exige contenerizar y desplegar a ECR, pero el patron es identico al que ya construiste: leer hallazgos → calcular score → decidir si el pipeline continua.

> ⚠️ Inspector v2 no escanea codigo fuente: escanea **dependencias instaladas y el OS**. Si tienes una vulnerabilidad en tu propio codigo (logica de negocio), Inspector no la ve. Para eso estan las herramientas de SAST (Static Application Security Testing) como CodeGuru o SonarQube.

A tener en cuenta

Lo que conecta la Sesion 2 con el resto del curso

Lo construido hoy

- Un **servicio bajo prueba** real con 3 suites de tests (productos, pedidos, contrato/SLA).
- Un **buildspec.yml** con las 4 fases de CodeBuild, versionado junto al codigo.
- Un **runner local** en Node que ejecuta el mismo pipeline en tu maquina, sin depender de AWS ni de PowerShell.
- Un **pipeline CodePipeline** que conecta push → build → test → deploy.
- Un **security gate con Inspector**: score de riesgo que puede bloquear el deploy.

Herramienta AWS	Alternativas equivalentes	Concepto comun
CodeBuild	GitHub Actions, Jenkins, CircleCI, GitLab CI	Build runner con buildspec/workflow
CodePipeline	GitHub Actions (workflow), Azure DevOps Pipelines	Orquestador de stages CI/CD
AWS Inspector	Snyk, Trivy, Aqua Security, Dependabot	Escaneo de vulnerabilidades en deps/OS
CloudWatch Logs	Datadog, Splunk, ELK Stack	Centralizacion de logs del pipeline

> 💡 Mensaje del dia: el pipeline no es una herramienta de DevOps. Es una herramienta de **calidad**. Cada test que pones en el pipeline es una regla de negocio que nadie puede romper sin que el equipo lo sepa en minutos.